

G큐브 외부 보드 (마이크로비트) 연결 매뉴얼

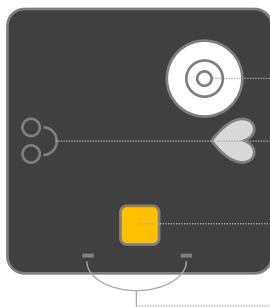
Open platform IoT robotics module



G-Cube

G큐브는 CPU, 모터, 배터리, 각종 센서가 내장된 IoT모듈로 G큐브 1개로부터 로봇을 만들 수 있고 여러 개를 블루투스로 연결하여 무한 확장하며 로봇틱스, AI, SW, 사물인터넷, 빅데이터 프로젝트에 활용할 수 있는 오픈 플랫폼입니다.

GCube

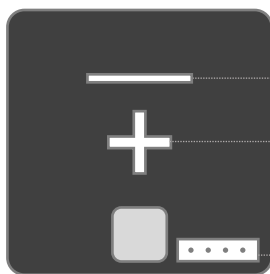


충전용 자석 단자 (상)

적외선 근접 센서

버튼

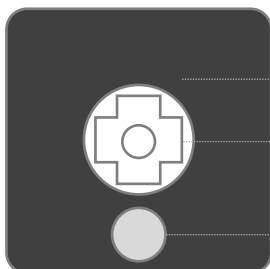
연결, 충전 표시등



LED 표시등

링크 홀

4핀 확장 포트
(GND, Vout, SCL, SDA)
(GND, Vout, TXD, RXD)



자이로센서, 부저

스텝 모터

도형 모양

GCube applications

G큐브

BLE5.0 CPU

배터리

스텝모터



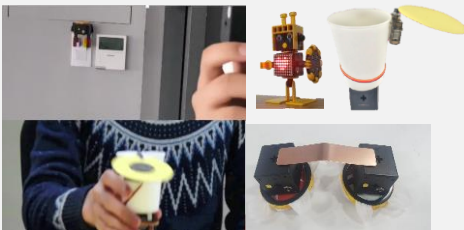
내장센서
[자이로/근접]
[버튼/부저]

외부센서포트

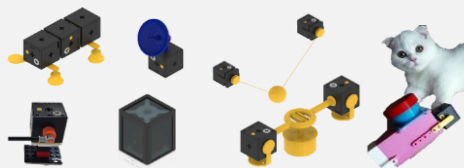
1 G큐브 + 링크 → 핑퐁 로봇



2 G큐브 + 센서 → IoT 네트워크



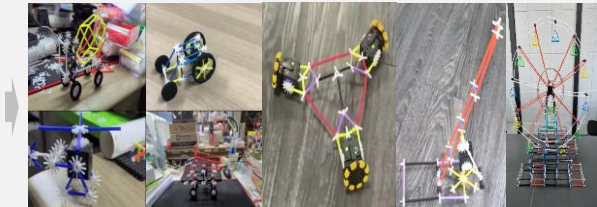
3 G큐브 + 3D 프린터 → 메이커



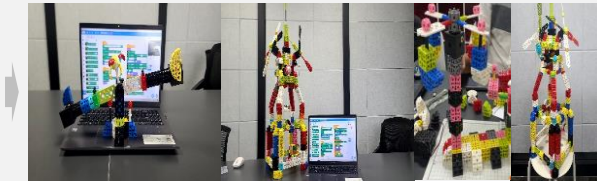
4 G큐브 + 레고 → 브릭큐브 로봇



5 G큐브 + 포디프레임 → 포디큐브로봇



6 G큐브 + 큐브웍스 → 큐브웍스 로봇



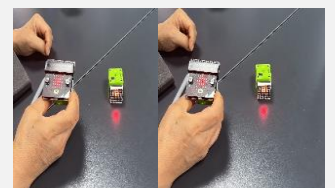
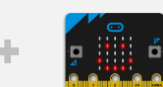
7 G큐브 + 케이넥스, MRT, 매직토이



8 G큐브 + 아두이노 → IoT 로봇틱스

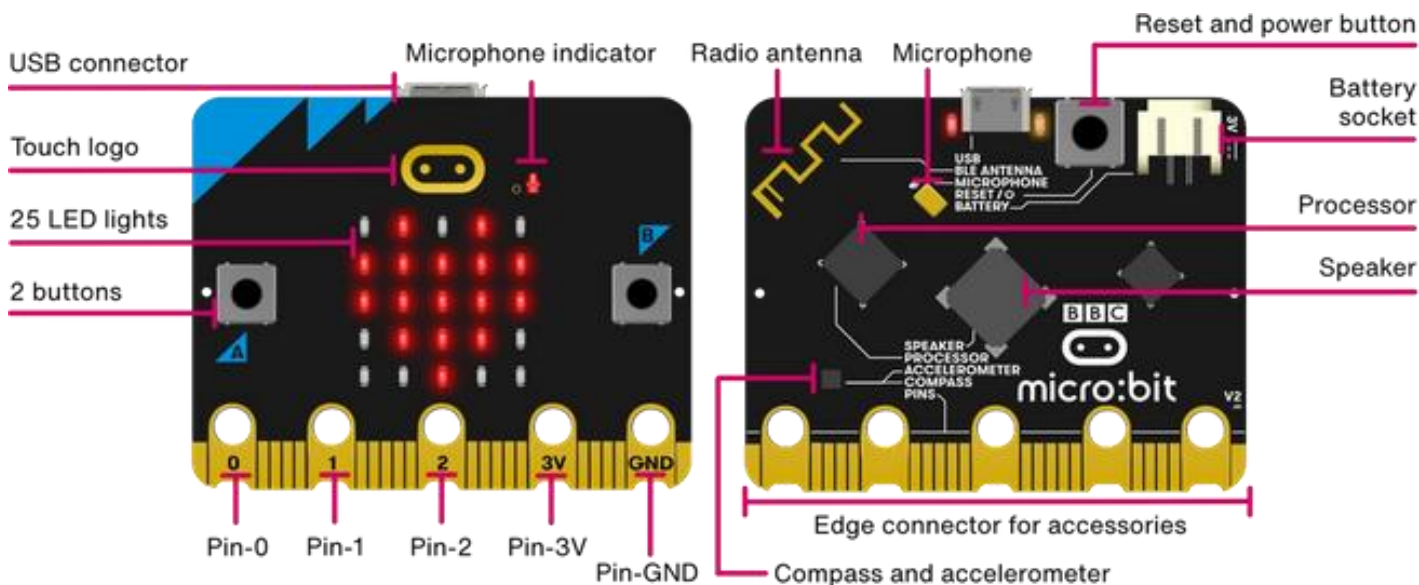
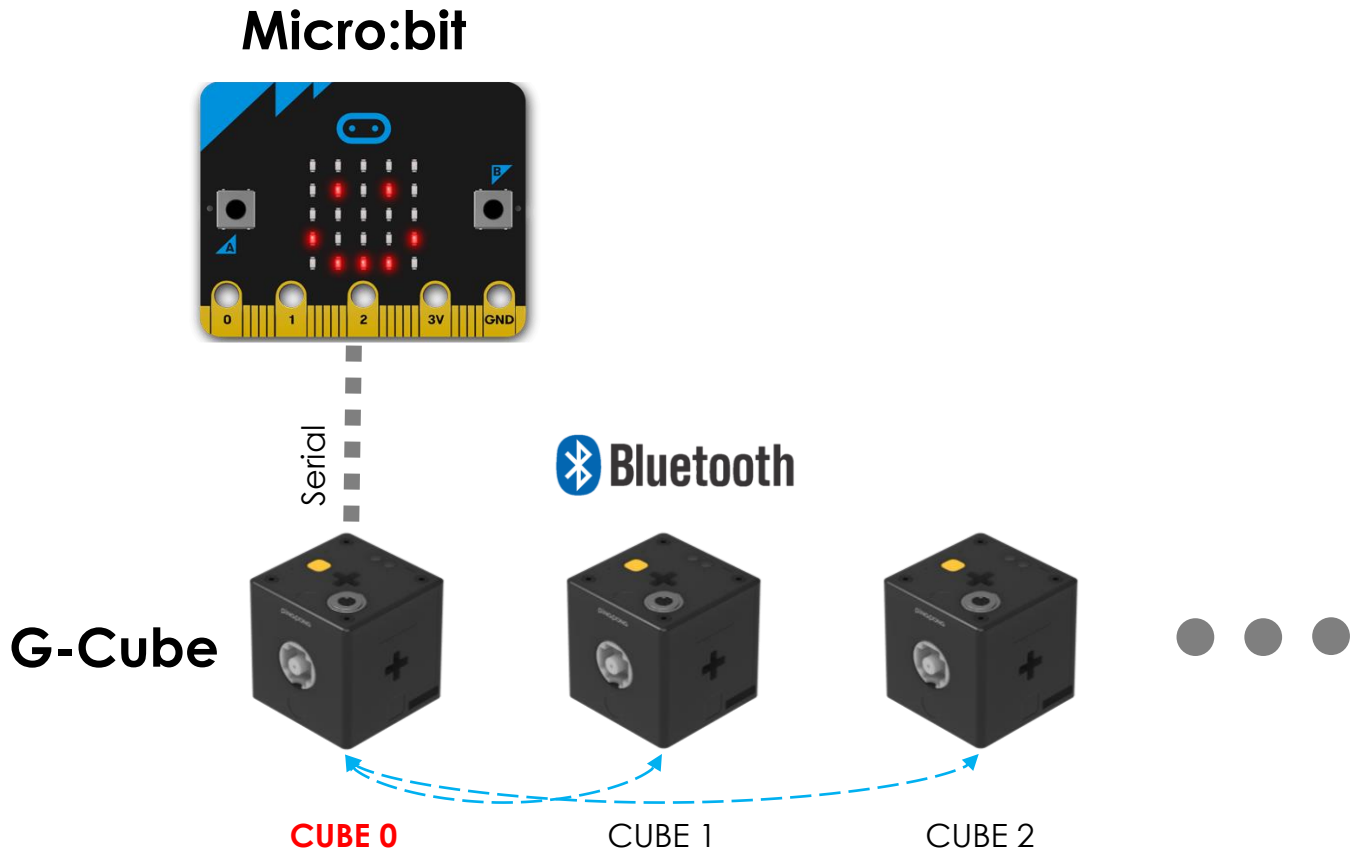


9 G큐브 + 마이크로비트 → SW 교육



Mission with Microbit

G큐브는 마이크로비트, 아두이노, 라즈베리파이 등의 보드와 시리얼통신으로 연결하여 보드를 사용하는 사용자가 G큐브를 활용한 로봇틱스와 사물인터넷 프로젝트를 쉽게 수행할 수 있도록 라이브러리 및 예제 프로그램을 제공합니다.



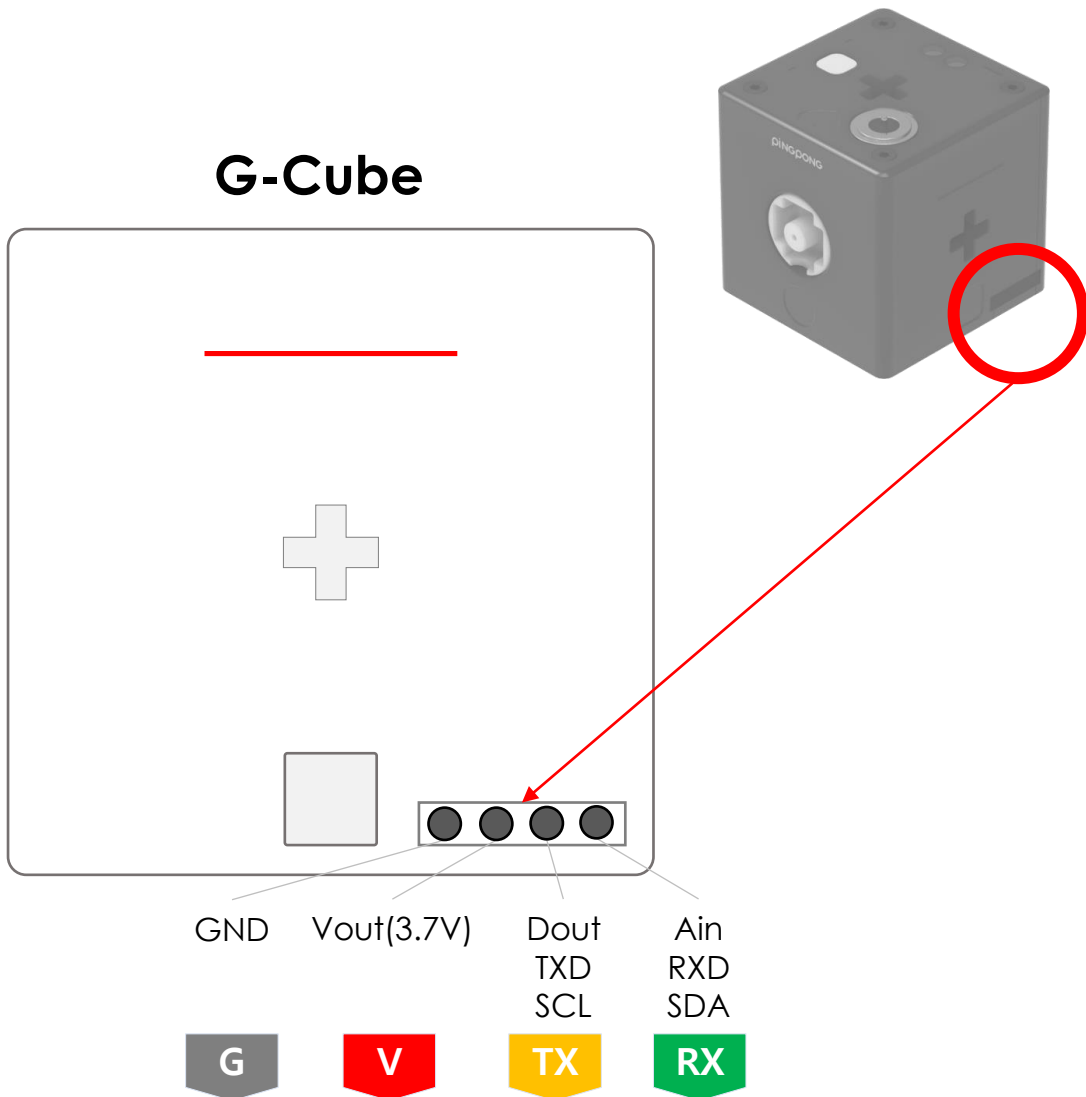
Pin Map (Serial)

G큐브와 외부 보드 (아두이노, 마이크로비트 등)를 연결할 때 G큐브 측면 하단에 아래와 외부 확장포트가 있습니다. 이 핀맵을 참조하여 보드에 점퍼 케이블로 연결해 줍니다.

Baudrate : 115200

Arduino pins : 2 (RX), 3 (TX), 5V, GND

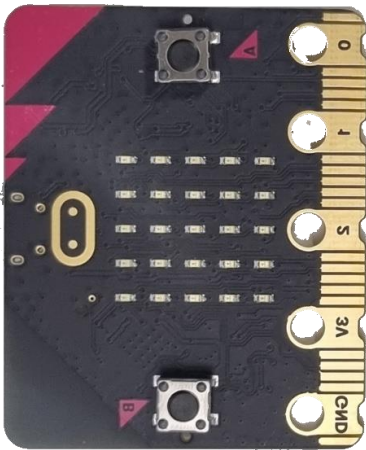
Microbit pins : P0 (RX), P1(TX), 3V, GND



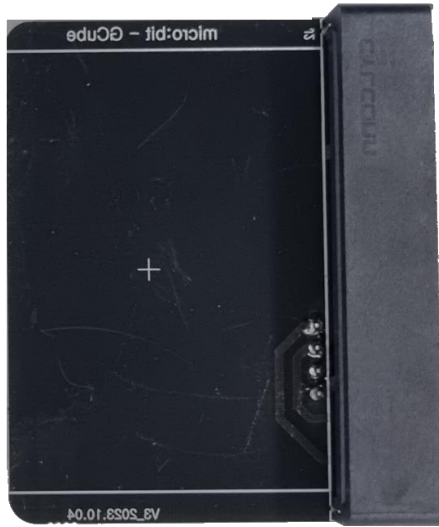
- Vout : 큐브 내부의 배터리 전원을 밖으로 연결해 둔 포트로 3.7V ~ 4.2V 값을 가진다
- GND : 큐브 내부의 Ground.
- Dout/TXD/SCL : Digital 출력핀, 시리얼통신 TX 핀, I2C의 SCL 핀으로 사용한다.
- Ain/RXD/SDA : Analog **입력핀**, 시리얼통신 RX 핀, I2C의 SDA 핀으로 사용한다.

Connection of Microbit and GCube

- 마이크로비트 + 엣지커넥터 연결



Microbit



Edge Connector



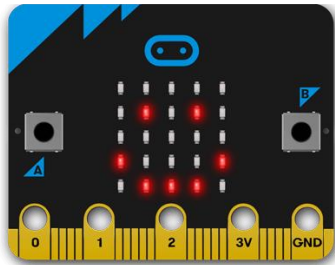
**Edge Connector
+ Microbit**



**Edge Connector
+ Microbit + GCube**

Connection of Microbit and GCube 0 ~

Micro:bit



- STEP 1 : G큐브-마이크로비트 예시 코드를 마이크로비트에 업로드
- STEP 2 : 어댑터를 이용하여 마이크로비트를 G큐브 확장포트에 꽂기
- STEP 3 : 마이크로비트 연결된 0번 G큐브 전원 ON 하면 보드와 자동연결
- STEP 4 : 사용자가 설정한 프로젝트에 필요한 큐브의 개수만큼 연결 되도록 나머지 G큐브 전원버튼 2번씩 누르면 큐브 끼리 블루투스 연결 완료
- STEP 5 : 필요한 큐브 연결이 끝나면 예시 프로그램 수행됨

Serial

Bluetooth



CUBE 0



CUBE 1



CUBE 2



- 보드가 연결된 0번 G큐브에 다른 G큐브들을 연결하려면 연결될 G큐브들이 모두 같은 그룹번호로 설정되어 있어야 한다.
- 그룹번호를 부여하는 방법은 Android/iOS 그룹위저드앱 (pingpong group wizard) 사용함 (YouTube 로보라이즌 채널 -> 교육동영상 참조)



CUBE 0



CUBE 1



CUBE 2



CUBE 3



CUBE 4



CUBE 5

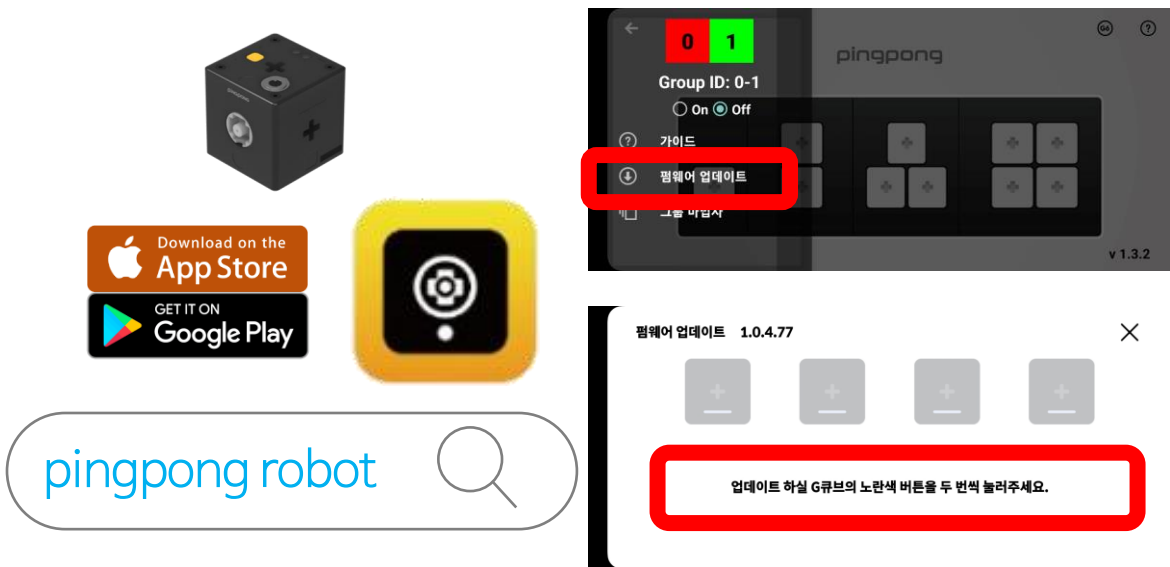
- 큐브 번호는 0번~7번까지 부여됨
- 0번큐브는 보드와 연결된 큐브이고 1~7번은 0번 큐브와 블루투스로 연결되는 큐브 (연결된 순서대로 번호(색상) 부여)
- 0번 G큐브는 빨간색 LED가 켜지고 나머지 큐브들은 연결된 순서에 따라 Green, Blue, Cyan, Magenta, Yellow, Violet, Orange 등의 순서로 켜진다.

GCube 펌웨어 확인 및 그룹번호 맞추기

펌웨어 업데이트

마이크로비트/아두이노에 G큐브를 연결하여 사용하려면 G큐브의 펌웨어 버전이 77 이상의 버전이어야 합니다.

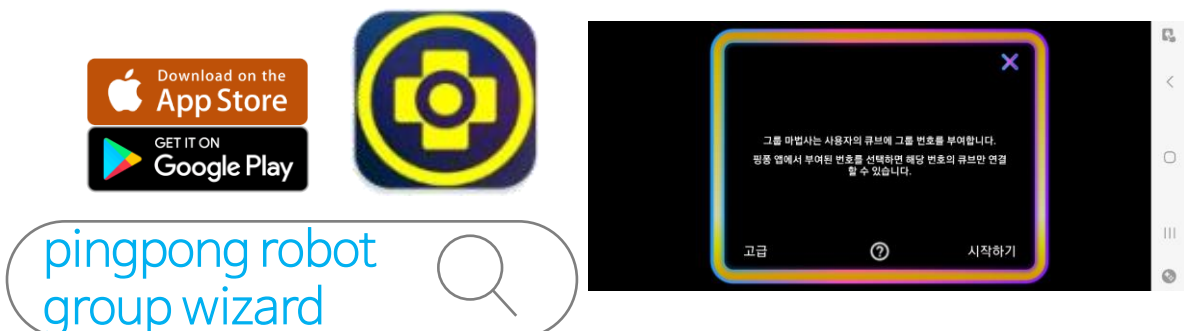
Google Play 또는 Apple store 에서 “pingpong robot” app을 설치하신 후 펌웨어 업데이트 화면에서 G큐브를 연결하여 펌웨어 업데이트를 진행합니다.



그룹번호 동일하게 지정하기

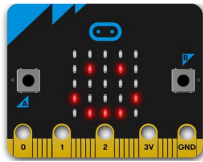
마이크로비트/아두이노에 G큐브를 연결하여 사용하려면 프로젝트에 사용할 G큐브의 그룹번호가 모두 동일하게 설정되어 있어야 합니다. 여러 명의 사람이 동시에 사용할 때 블루투스 연결 혼선을 막기 위해 동일한 그룹번호를 가진 G큐브들이 서로 연결됩니다.

- 그룹번호를 부여하는 방법은 Android/iOS 그룹위저드앱 (pingpong group wizard) 사용함 (YouTube 로보라이즌 채널 -> 교육동영상 참조)



GCube 와 연결하고 실행하기

STEP 1 : 마이크로비트에 (GCube-Microbit) 예시 코드를 업로드 합니다



STEP 2 : 마이크로비트를 어댑터에 꽂고 어댑터를 G큐브 확장포트에 꽂습니다.

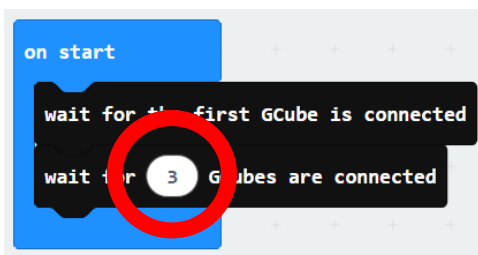


STEP 3 : 마이크로비트가 꽂혀 있는 G큐브 (0번큐브)
의 노랑 버튼을 **한 번** 눌러줍니다.

*마이크로비트에 예시코드가 정상적으로 업로드 된 경우 연결음이 나고 마이크로비트 LED 매트릭스의 중앙 (2,2) LED가 켜집니다. 이 때 G큐브 옆면 LED도 빨강색으로 깜빡이며 켜집니다.



STEP 4 : 다른 G큐브의 노란색 버튼을 **두 번씩** 눌러줍니다. 연결되는 순서에 따라 초록색, 파랑색, 하늘색 ... 순서로 G큐브 옆면 LED가 켜집니다. 사용자가 마이크로비트에서 설정한 숫자만큼의 큐브 (0번큐브 포함)가 모두 연결되면 마이크로비트 상단의 LED가 박스 형태로 켜집니다.

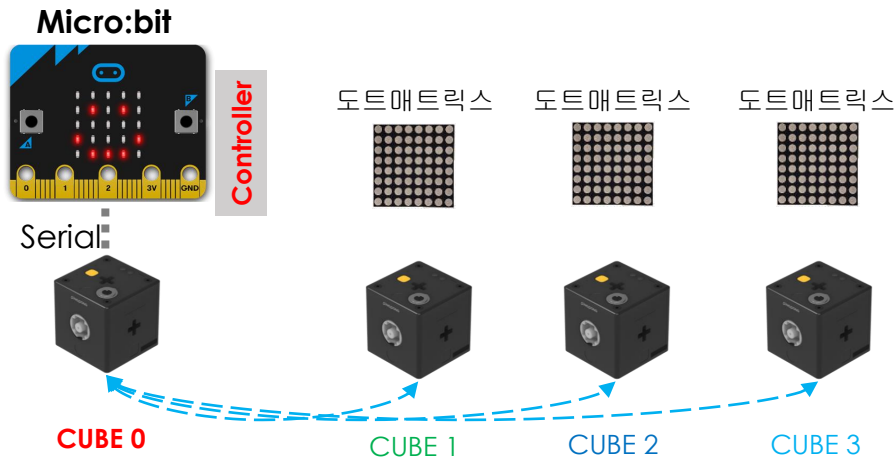


CUBE 0

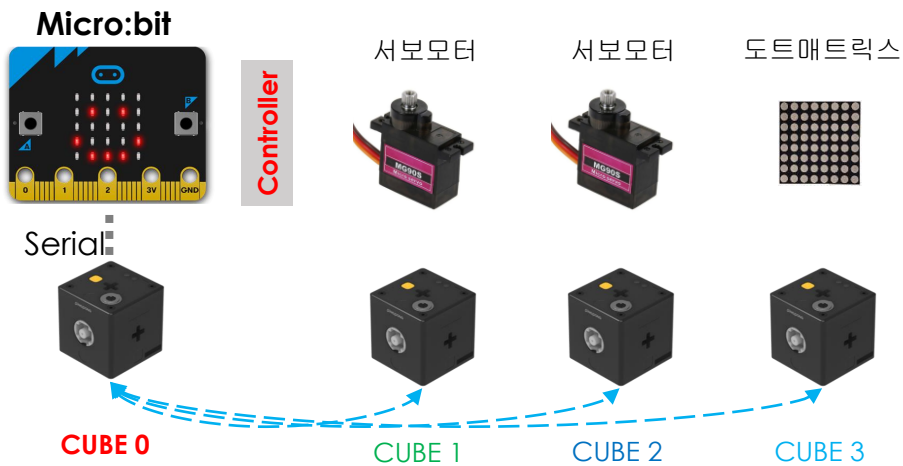
CUBE 1

CUBE 2

Network configuration examples (2)

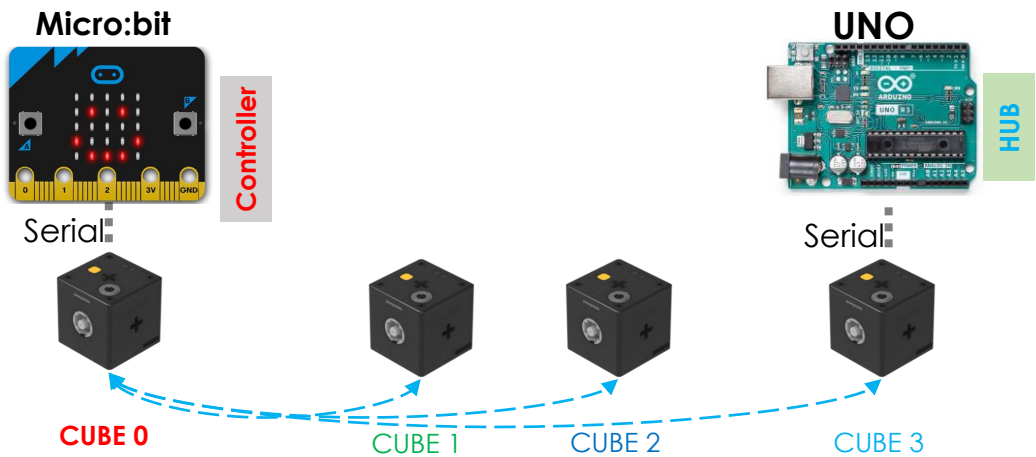
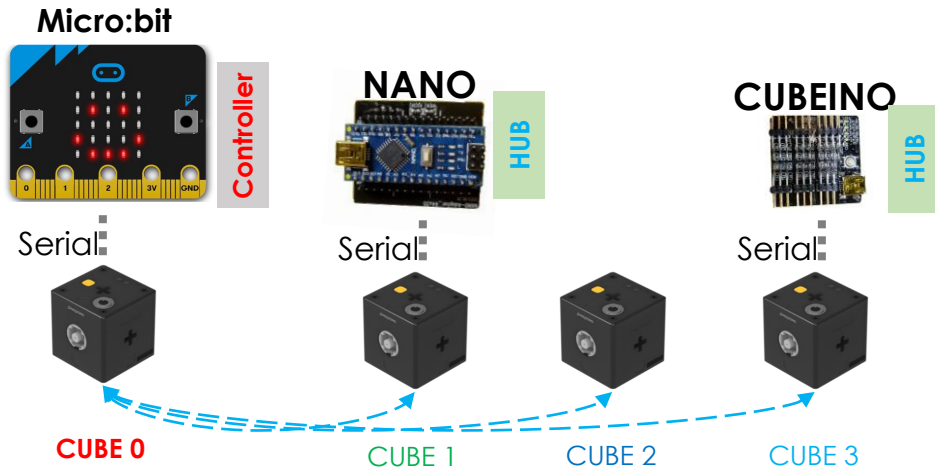


- GCube 0 에 마이크로비트를 연결하고 GCube 0번을 켜다.
- GCube 1, 2, 3에 도트매트릭스를 켜둔 상태에서 전원 버튼을 두 번 눌러 GCube 0에 연결한다.
- 마이크로비트 예시 프로그램(Matrix Art 등)으로 아이콘/애니메이션을 매트릭스에 표현한다.

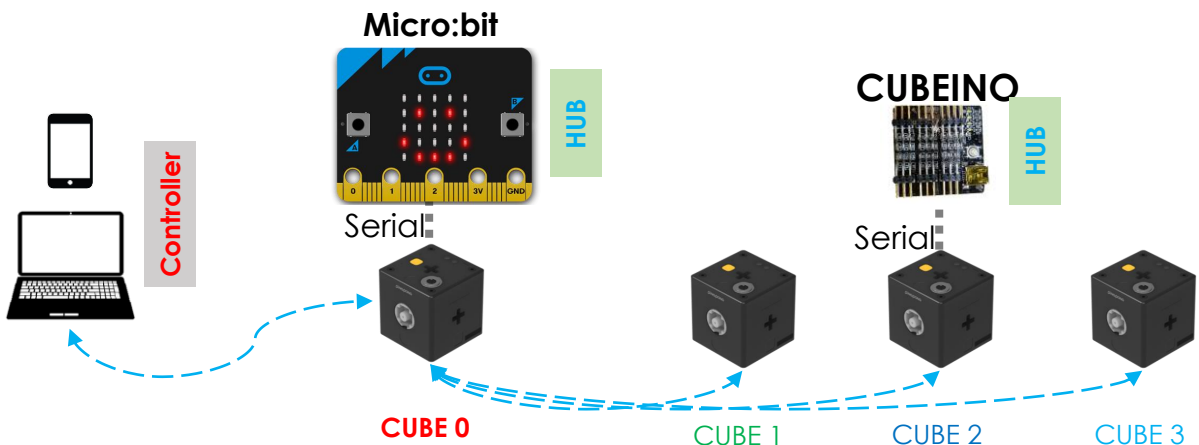


- GCube 0 에 마이크로비트를 연결하고 GCube 0번을 켜다.
- GCube 1, 2는 서보모터를, 3에는 도트매트릭스를 켜둔 상태에서 전원 버튼을 두 번 눌러 GCube 0에 연결한다.
- 마이크로비트 예시 프로그램을 참조하여 서보모터와 도트매트릭스를 제어한다.

Network configuration examples (1)

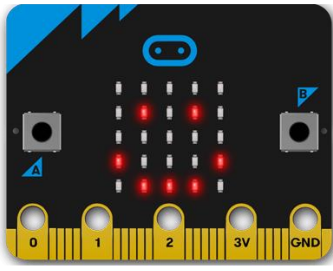


- 각 예시에서 왼쪽 마이크로비트에서 GCube 0~3의 모터 등을 제어할 수 있도록 메이크코드로 코딩한다.
- 각 예시에서 왼쪽 마이크로비트에서 Gcube 1~3의 각 센서 및 각 Gcube에 연결되어 있는 아두이노 보드의 아날로그 값을 가져오거나 디지털 포트의 On/Off를 메이크코드에서 코딩한다.
- 위 그림에서 HUB 표시된 보드에는 라이브러리 예시 sensorHub.ino 프로그램이 업로드 되어 있어야 한다.

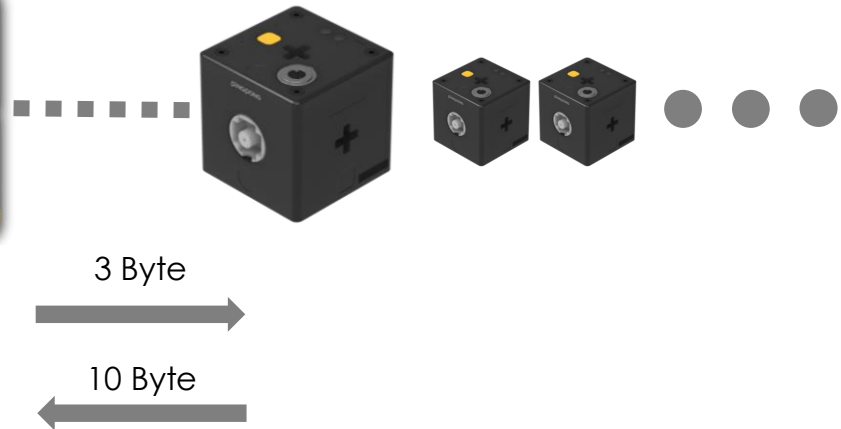


- 위 예시에서 왼쪽 스마트폰/노트북/크롬북 등에서 GCube 0~3를 블루투스와 연결하고 모터, 센서 등을 제어할 수 있도록 스크래치, 엔트리, 파이썬을 이용해 코딩한다.
- 위 예시에서 왼쪽 스마트폰/노트북/크롬북 등에서 GCube 0~3의 각 센서 및 각 Gcube에 연결되어 있는 아두이노 보드의 아날로그 값을 가져오거나 디지털 포트를 제어하도록 스크래치, 엔트리, 파이썬을 이용해 코딩한다.

Protocol – type A



센서허브역할



G큐브 -> 보드 [3 Byte] : REQ

보드 -> G큐브 [10 Byte] : RCV (G큐브에서 물어본 것에 대한 대답), CMD (G큐브에 행동할 것을 명령함)

[G큐브에서 보드를 센서보드로 사용할 때]

*Zx는 임의의 숫자를 의미함, 보통 0을 입력하여 사용함

*IV 는 프로토콜 안정성 목적으로 명령어와 반대 순서 숫자 (예: REQ(0x12) -> IV(0x21)) 를 넣어줌

1. G큐브-Arduino 아날로그 값 주기적 전송

REQ(0x11) + period + bitFlag // G큐브에서 아두이노로 요구 명령 음 (한번만)

RCV(0x11) + IV + A3+ A2 + A1_hi + A1_lo + A0_hi + A0_lo + Z[~] // 주기적으로 아두이노 보드 아날로그값 읽어서 G큐브로 보내줌

// bitFlag : 0000xxxx A3~A0 포트On/Off 비트 플래그

// period : 아날로그 읽는 주기 설정 (1~100, No response) // 1:100msec 100:10sec

2. G큐브-Arduino 디지털 포트 제어

REQ(0x1A) + PORT[H] + PORT[L] // 각 디지털 포트별 On/Off (Bit4:P4 ~ Bit13:P13)

3. 연결된 큐브 센서 값, 큐브-큐브이노(아두이노) 센서값 읽기

CMD(0xA0+CubeNumber) + IV + Z[~] // CubeNumber: GCube 0~7, GCube N 의 기울기 센서값 읽기 명령

REQ(0xA0+CubeNumber) + X + Y // GCube N 의 기울기 값 회신 (X, Y : -90~90), GCube N 의 기울기 센서값 읽기 데이터

CMD(0xAA) + IV + CubeNumber + Z[~] // CubeNumber: GCube 0~7, GCube N 의 근접 센서값 읽기 명령

REQ(0xAA) + CubeNumber + P // GCube N 의 근접 센서값 읽기 데이터

CMD(0xAB) + IV + CubeNumber + Z[~] // CubeNumber: GCube 0~7, GCube N 의 버튼의 상태값(눌림/떼짐) 읽기 명령

REQ(0xAB) + CubeNumber + B // GCube 버튼의 상태값 읽기 데이터

CMD(0xB0+CubeNumber) + IV + Z[~] //CubeNumber : GCube 0~7, GCube N 의 확장포트 센서값 읽기 명령

REQ(0xB0+CubeNumber) + Ext_hi + Ext_lo // GCube N 의 확장포트 센서값 읽기 데이터

//초음파(거리)센서 : 8bit(Ext_hi)

//소리,빛,볼륨,자기력 : 12bit (Ext_hi*16+Ext_lo) or 8bit(Ext_lo)

//온도센서 : 8bit+2bit(Ext_hi : 소숫점 위, Ext_lo : 소숫점 아래)

//칼라센서 : REQ Header(Red), Ext_hi(Green), Ext_lo(Blue)

CMD(0xE8+CubeNumber) + IV + Z[~] //CubeNumber : GCube 0~7, GCube N 의 확장포트-칼라센서값 읽기 명령

REQ(Red) + Green + Blue // GCube N 의 확장포트-칼라센서값 읽기 데이터

CMD(0xD0+CubeNumber)+ IV + A[3~0] + Z[~] //CubeNumber : GCube 0~7, A3 ~ A0 : 3 ~ 0 //GCube N의 Ax 값 읽기 명령

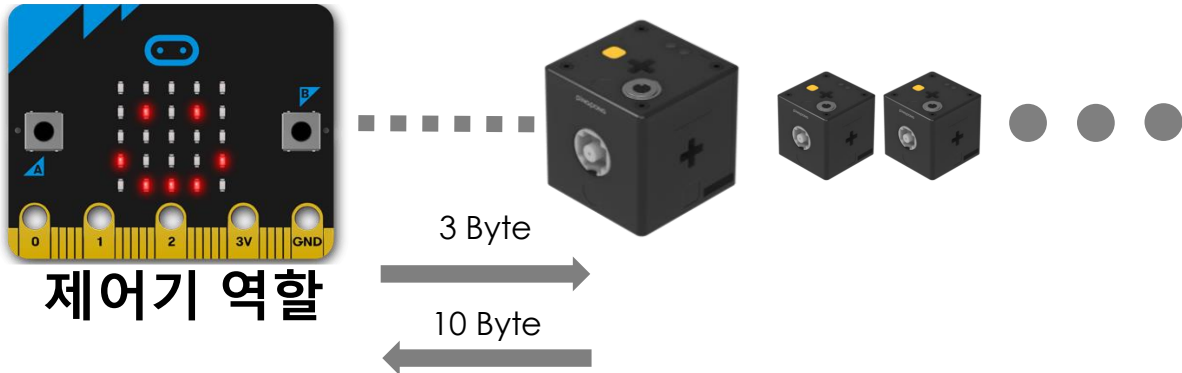
REQ(0xD0+CubeNumber) + A_hi + A_lo // 12 bit 이므로 Value = A_hi * 256 + A_lo //GCube N의 Ax 값 읽기 결과

*A3, A2, A1의 경우는 8bit(1바이트) 분해능 이므로 A_lo로 데이터 보내고 A_hi = 0을 넣어줌..

CMD(0xD8) + IV + CubeNumber + Z[~] //CubeNumber 의 아날로그 센서 읽기 시작 모드 전송

CMD(0xD9) + IV + CubeNumber +PORT[H] + PORT[L] + Z[~] // 특정 큐브-아두이노의 디지털 포트 제어 명령

Protocol – type B(1/2)



[보드에서 G큐브를 제어하는 프로젝트로 사용할 때]
 *Zx는 임의의 숫자를 의미함, 보통 0을 입력하여 사용함
 *IV는 프로토콜 안정성 목적으로 명령어와 반대 순서 숫자 (예: REQ(0x12) -> IV(0x21))를 넣어줌

1. 보드 ID 확인

REQ(0x10) + Z1 + Z0
 RCV(0x10) + IV + VID1 + VID0 + BID1 + BID0 + Z[~] (Vendor ID 2Byte + Board ID 2 Byte)

2. 연결된 큐브 개수 확인 및 연결완료 알려줌

CMD(0x21) + IV + Z[~] // STAR 큐브에 연결된 큐브 개수 확인해 줄것을 큐브에 요청
 REQ(0x21) + Z[1] + N_CUBE (Number of Cube) // STAR 큐브에서 연결된 큐브 회신 함
 CMD(0x2A) + IV + TOTAL_CUBE + Z[~] // STAR 큐브에 모든 큐브가 연결완료됨을 알려줌

3. 모터 회전 (스피드, 회전각도)

CMD(0x30) + IV + CubeNum + Speed + Z[~] // Set speed of a Cube (Speed : -100~100) //특정 번호 큐브 모터의 속도명령 전송
 CMD(0x31) + IV + 0 + 0 + 0 + 0 + 0 + 0 + s1 + s0 // Set speed of Cube 0 ~ 1 (sx : -100~100) (0~1)큐브까지의 속도명령 전송
 CMD(0x32) + IV + 0 + 0 + 0 + 0 + 0 + 0 + s2 + s1 + s0 // Set speed of Cube 0 ~ 2 (sx : -100~100) (0~2)큐브까지의 속도명령 전송
 CMD(0x33) + IV + 0 + 0 + 0 + 0 + 0 + 0 + s3 + s2 + s1 + s0 // Set speed of Cube 0 ~ 3 (sx : -100~100) (0~3)큐브까지의 속도명령 전송
 CMD(0x34) + IV + 0 + 0 + 0 + 0 + 0 + 0 + s4 + s3 + s2 + s1 + s0 // Set speed of Cube 0 ~ 4 (sx : -100~100) (0~4)큐브까지의 속도명령 전송
 CMD(0x35) + IV + 0 + 0 + 0 + 0 + 0 + 0 + s5 + s4 + s3 + s2 + s1 + s0 // Set speed of Cube 0 ~ 5 (sx : -100~100) (0~5)큐브까지의 속도명령 전송
 CMD(0x36) + IV + 0 + 0 + 0 + 0 + 0 + 0 + s6 + s5 + s4 + s3 + s2 + s1 + s0 // Set speed of Cube 0 ~ 6 (sx : -100~100) (0~6)큐브까지의 속도명령 전송
 CMD(0x37) + IV + 0 + 0 + 0 + 0 + 0 + 0 + s6 + s5 + s4 + s3 + s2 + s1 + s0 // Set speed of Cube 0 ~ 7 (sx : -100~100) (0~7)큐브까지의 속도명령 전송

CMD(0x38) + IV + CubeNum + rxh + rxl + Z[~] // Set angle of a Cube (rxh/rxl : -10000~10000) //특정 번호 큐브 모터의 회전각도 명령 전송

CMD(0x39) + IV + 0 + 0 + 0 + 0 + 0 + 0 + r1h + r1l + r0h + r0l // Set angle of Cube 0 ~ 1 (rxh/rxl : -10000~10000) //0~1번 큐브 모터의 회전각도 명령 전송

CMD(0x3A) + IV + 0 + 0 + 0 + 0 + 0 + 0 + r2h + r2l + r1h + r1l + r0h + r0l // Set angle of Cube 0 ~ 2 (rxh/rxl : -10000~10000) //0~2번 큐브 모터의 회전각도 명령 전송

CMD(0x3B) + IV + 0 + 0 + 0 + 0 + 0 + 0 + r3h + r3l + r2h + r2l + r1h + r1l + r0h + r0l // Set angle of Cube 0 ~ 3 (rxh/rxl : -10000~10000) //0~3번 큐브 모터의 회전각도 명령 전송

CMD(0x3C) + IV + 0 + 0 + 0 + 0 + 0 + 0 + r5h + r5l + r4h + r4l // Set angle of Cube 4 ~ 5 (rxh/rxl : -10000~10000) //4~5번 큐브 모터의 회전각도 명령 전송

CMD(0x3D) + IV + 0 + 0 + 0 + 0 + 0 + 0 + r6h + r6l + r5h + r5l + r4h + r4l // Set angle of Cube 4 ~ 6 (rxh/rxl : -10000~10000) //4~6번 큐브 모터의 회전각도 명령 전송

CMD(0x3E) + IV + 0 + 0 + 0 + 0 + 0 + 0 + r7h + r7l + r6h + r6l + r5h + r5l + r4h + r4l // Set angle of Cube 4 ~ 7 (rxh/rxl : -10000~10000) //4~7번 큐브 모터의 회전각도 명령 전송

Protocol – type B(2/2)

4. 서보 모터 각도 제어

CMD(0x40) + IV + CubeNum + a0 + Z[~] // Set servo angle of a Cube : (ax : 0~180) //특정 번호 큐브에 장착된 서보모터 각도 제어

CMD(0x41) + IV + 0 + 0 + 0 + 0 + 0 + 0 + a1 + a0 // Set speed of Cube 0 ~ 1 (ax : 0~180)

(0~1) 큐브_서보모터의 속도명령전송

CMD(0x42) + IV + 0 + 0 + 0 + 0 + 0 + a2 + a1 + a0 // Set speed of Cube 0 ~ 2 (ax : 0~180)

(0~2) 큐브_서보모터의 각도명령전송

CMD(0x43) + IV + 0 + 0 + 0 + 0 + a3 + a2 + a1 + a0 // Set speed of Cube 0 ~ 3 (ax : 0~180)

(0~3) 큐브_서보모터의 각도명령전송

CMD(0x44) + IV + 0 + 0 + 0 + a4 + a3 + a2 + a1 + a0 // Set speed of Cube 0 ~ 4 (ax : 0~180)

(0~4) 큐브_서보모터의 각도명령전송

CMD(0x45) + IV + 0 + 0 + a5 + a4 + a3 + a2 + a1 + a0 // Set speed of Cube 0 ~ 5 (ax :

0~180) (0~5) 큐브_서보모터의 각도명령전송

CMD(0x46) + IV + 0 + a6 + a5 + a4 + a3 + a2 + a1 + a0 // Set speed of Cube 0 ~ 6 (ax :

0~180) (0~6) 큐브_서보모터의 각도명령전송

CMD(0x47) + IV + a7 + a6 + a5 + a4 + a3 + a2 + a1 + a0 // Set speed of Cube 0 ~ 7 (ax :

0~180) (0~7) 큐브_서보모터의 각도명령전송

* 0번 큐브 (STAR)에 서보모터 명령어가 주어지면 큐브이노에 붙어 있는 서보모터 (D3 PWM이용)를 제어함 (핑퐁 모노용)

5. 도트매트릭스

CMD(0x51) + IV + r7 + r6 + r5 + r4 + r3 + r2 + r1 + r0 (DotMatrix data of Cube 1) //1번 큐브에 장착된 도트매트릭스 화면

CMD(0x52) + IV + r7 + r6 + r5 + r4 + r3 + r2 + r1 + r0 (DotMatrix data of Cube 2) //2번 큐브에 장착된 도트매트릭스 화면

CMD(0x53) + IV + r7 + r6 + r5 + r4 + r3 + r2 + r1 + r0 (DotMatrix data of Cube 3) //3번 큐브에 장착된 도트매트릭스 화면

CMD(0x54) + IV + r7 + r6 + r5 + r4 + r3 + r2 + r1 + r0 (DotMatrix data of Cube 4) //4번 큐브에 장착된 도트매트릭스 화면

CMD(0x55) + IV + r7 + r6 + r5 + r4 + r3 + r2 + r1 + r0 (DotMatrix data of Cube 5) //5번 큐브에 장착된 도트매트릭스 화면

CMD(0x56) + IV + r7 + r6 + r5 + r4 + r3 + r2 + r1 + r0 (DotMatrix data of Cube 6) //6번 큐브에 장착된 도트매트릭스 화면

CMD(0x57) + IV + r7 + r6 + r5 + r4 + r3 + r2 + r1 + r0 (DotMatrix data of Cube 7) //7번 큐브에 장착된 도트매트릭스 화면

* 0번 큐브 (STAR)에는 도트매트릭스를 연결할 수 없음

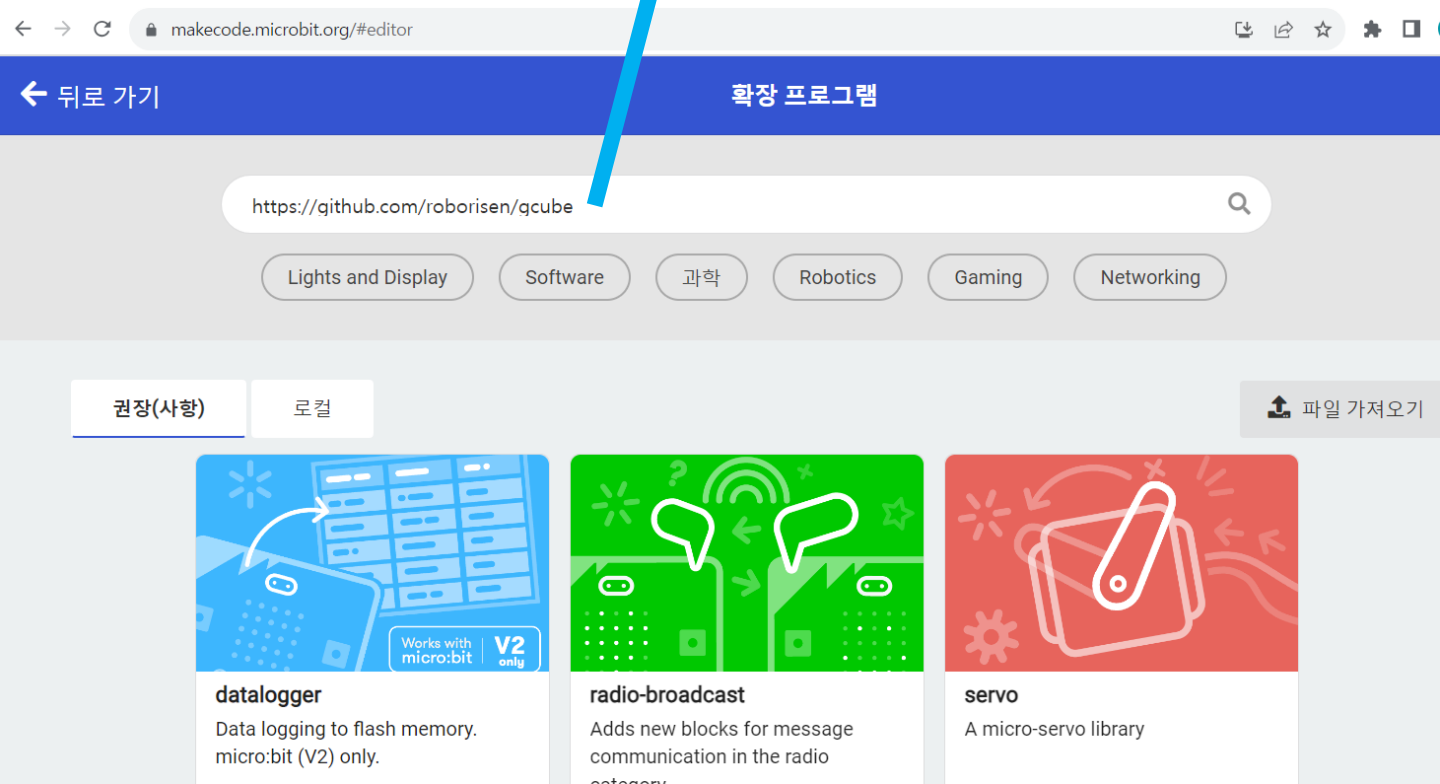
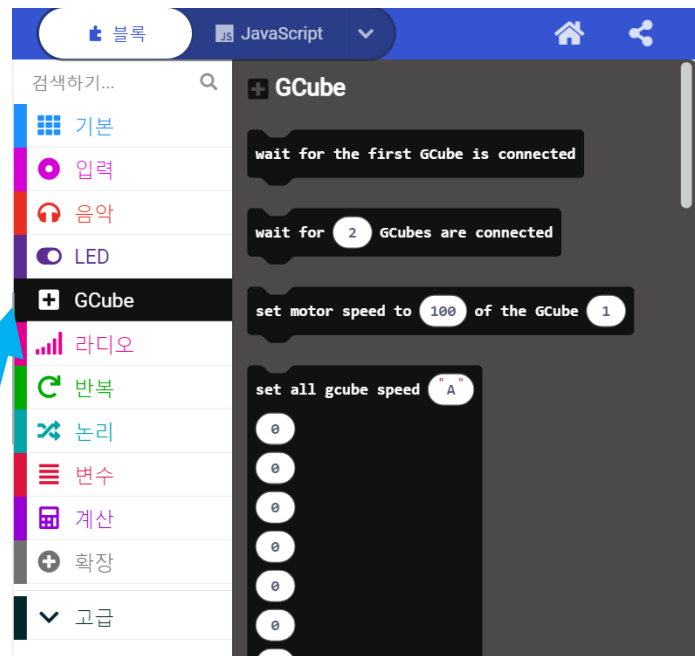
GCube 확장 블록 가져오기

How to connect Micro:bit to GCube

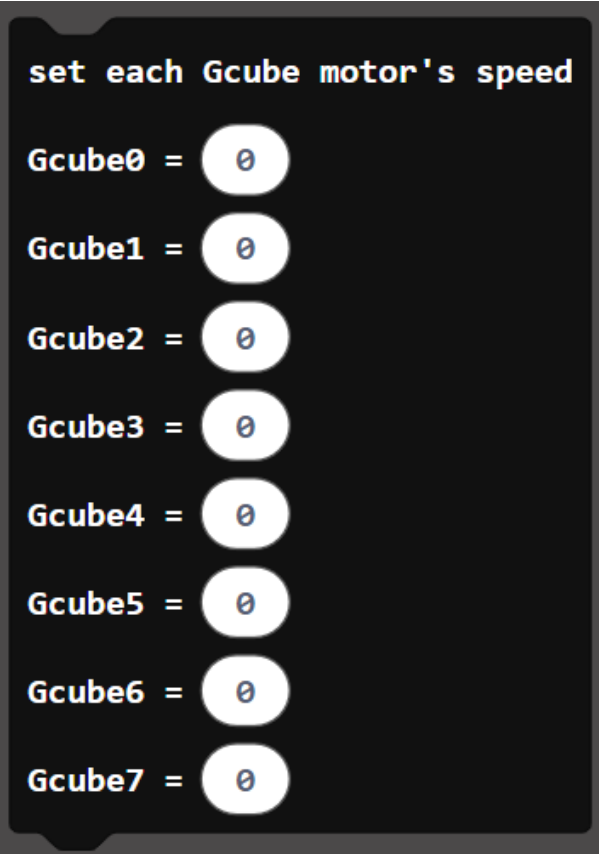
- * Step 1: Load program to Micro:bit using this GCube custom blocks
- * Step 2: Attach Micro:bit to GCube using GCube-Edge connector module
- * Step 3: Turn on the GCube of Step 2 (for dot-matrix application, you should attach dot matrix to GCube in advance)
- * Step 4: Click twice of the other GCubes (to defined number at Step 1 user program)
- * Step 5: If all GCubes are connected then the main mission of user program will be started

Micro:bit GCube block URL

- 1 <https://makecode.microbit.org> 열기
- 2 새로운 프로젝트**를 클릭하여 프로젝트 만들기
- 3 톱니바퀴 모양 메뉴에서 **확장**을 클릭합니다
=> "<https://github.com/roborisen/GCube>" 로
검색하고 가져오기



GCube 확장 블록 설명(1)

	보드에 연결된 0번 G큐브의 전원이 켜질 때까지 기다리기
	프로젝트에 필요한 N 개 G큐브가 모두 연결될 때까지 기다리기
	N번 G큐브의 모터 속도를 00 으로 정하기
	연결된 모든 G큐브의 모터를 멈추기
 <pre>set each Gcube motor's speed Gcube0 = 0 Gcube1 = 0 Gcube2 = 0 Gcube3 = 0 Gcube4 = 0 Gcube5 = 0 Gcube6 = 0 Gcube7 = 0</pre>	연결된 모든 G큐브의 모터의 속도를 정하기

GCube 확장 블록 설명(2)

set rotation angle to 90 of the GCube 1

N번 G큐브 모터의 회전 각도를 00도로 정하기

set each Gcube rotation angle

Gcube0 = 0

Gcube1 = 0

Gcube2 = 0

Gcube3 = 0

Gcube4 = 0

Gcube5 = 0

Gcube6 = 0

Gcube7 = 0

연결된 모든 G큐브 모터의 회전 각도를 각각 정하기

set servo motor angle to 90 of the GCube 1

N번 G큐브에 연결된 서보모터의 각도를 00도로 정하기

set each Gcubes servo motor angle

Gcube0 = 0

Gcube1 = 0

Gcube2 = 0

Gcube3 = 0

Gcube4 = 0

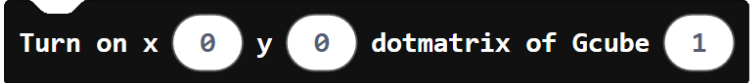
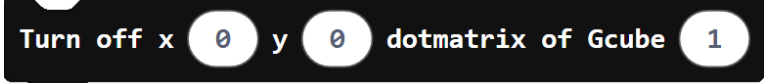
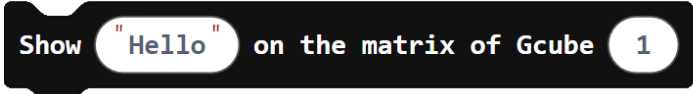
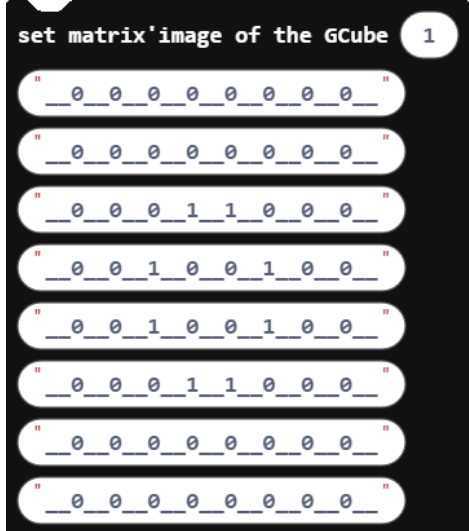
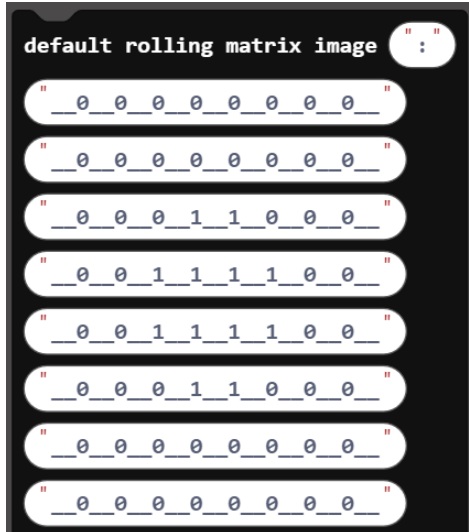
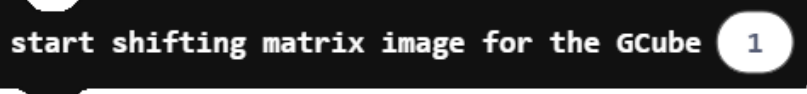
Gcube5 = 0

Gcube6 = 0

Gcube7 = 0

연결된 모든 G큐브에 연결된 서보모터의 각도를 각각 정하기

GCube 확장 블록 설명(3)

	N번 큐브에 꽂혀 있는 도트매트릭스의 x, y 픽셀 켜기
	N번 큐브에 꽂혀 있는 도트매트릭스의 x, y 픽셀 끄기
	N번 큐브에 꽂혀 있는 도트매트릭스에 문자열 보여주기
 <pre> set matrix'image of the GCube 1 "__0__0__0__0__0__0__0__0__" "__0__0__0__0__0__0__0__0__" "__0__0__0__1__1__0__0__0__" "__0__0__1__0__0__1__0__0__" "__0__0__1__0__0__1__0__0__" "__0__0__0__1__1__0__0__0__" "__0__0__0__0__0__0__0__0__" "__0__0__0__0__0__0__0__0__" </pre>	N번 G큐브에 연결된 8x8 도트매트릭스에 보여줄 기본 이미지를 이와 같이 정하기 - 는 해당위치 LED 꺼짐 # 은 해당위치 LED 켜짐 #, - 부분만 수정하여 원하는 이미지를 그리기
 <pre> default rolling matrix image ":" "__0__0__0__0__0__0__0__0__" "__0__0__0__0__0__0__0__0__" "__0__0__0__1__1__0__0__0__" "__0__0__1__1__1__1__0__0__" "__0__0__1__1__1__1__0__0__" "__0__0__0__1__1__0__0__0__" "__0__0__0__0__0__0__0__0__" "__0__0__0__0__0__0__0__0__" </pre>	N번 G큐브에 스크롤 할 기본 이미지 정하기
	스크롤(롤링) 이미지를 00초 동안 실행
	N 번 G큐브에 붙어있는 8x8 도트매트릭스 이미지를 마이크로비트의 기울기 센서값에 동기시켜 움직임. 번호가 0이면 연결된 모든 G큐브에 붙어 있는 도트매트릭스에 동시에 표현

GCube 확장 블록 설명(4)

proximitysensor ▼ value of GCube 1	N번 G큐브의 근접센서 또는 버튼센서 값
xdata ▼ value of GCube 1	N번 G큐브의 가속도 센서 값 (X 또는 Y)
external light ▼ sensor value of Gcube 1	N번 G큐브에 꽂혀 있는 (빛, 소리, 자석, 볼륨, 온도, 초음파, 아날로그) 센서의 값
color redcolor ▼ value of Gcube 1	N번 G큐브에 꽂혀 있는 칼라 센서의 R, G, B 값과 칼라키 값
Arduino 0 port value of GCube 1	N번 G큐브에 꽂혀 있는 아두이노의 A0~A3 포트에 연결된 센서의 값
move PingPong robot 10 cm for autocar ▼	선택된 핑퐁로봇모델(AucoCar, DrawingBot, BattleBot, AntBot)을 X cm 이동시키기
rotate PingPong robot 90 degree for autocar ▼	선택된 핑퐁로봇모델(AucoCar, DrawingBot, BattleBot, AntBot)을 X degree 회전시키기
penup ▼ of Drawing Bot	DrawingBot 로봇 모델의 펜 올리기/내리기
leverup ▼ of Battle Bot	BattleBot 로봇 모델의 레버 올리기/내리기
gripperopen ▼ of Ant Bot	AntBot 로봇 모델의 집게 닫기/열기

Example1 –GCube connection

예제 설명

1. 보드에 연결된 0번 G큐브가 켜질 때 까지 (전원버튼 1번 누름) 기다린다.
2. 나머지 1번, 2번 G큐브가 연결될 때 (1, 2번 큐브의 전원버튼 2번 누름) 까지 기다린다.

on start

wait until the first Gcube is connected

wait until 2 Gcubes are connected

Example2 – dotmatrix rolling

예제 설명

1. 보드에 연결된 0번 G큐브가 켜질 때 까지 (전원버튼 1번 누름) 기다린다.
2. 나머지 1번, 2번 G큐브에 도트매트릭스를 각각 켜는다.
3. 1, 2 번 G큐브가 연결될 때 (각 전원버튼 2번 누름) 까지 기다린다.
4. 마이크로비트의 A 버튼을 누르면 디폴트 롤링 이미지가 설정된다.
5. 마이크로비트의 B 버튼을 누르면 롤링 이미지가 각 큐브의 도트매트릭스를 넘어다니며 흐른다.

on start

wait until the first Gcube is connected

wait until 3 Gcubes are connected

on button B pressed

start rolling matrix image for 1000 seconds

on button A pressed

default rolling matrix image :

" _0_0_0_0_0_0_0_0_ "

" _0_0_0_0_0_0_0_0_ "

" _0_0_0_1_1_0_0_0_ "

" _0_0_1_1_1_1_0_0_ "

" _0_0_1_1_1_1_0_0_ "

" _0_0_0_1_1_0_0_0_ "

" _0_0_0_0_0_0_0_0_ "

" _0_0_0_0_0_0_0_0_ "

Example3 – dotmatrix shifting

예제 설명

1. 보드에 연결된 0번 G큐브가 켜질 때 까지 (전원버튼 1번 누름) 기다린다.
2. 나머지 1번, 2번 G큐브에 도트매트릭스를 각각 켜는다.
3. 1, 2 번 G큐브가 연결될 때 (각 전원버튼 2번 누름) 까지 기다린다.
4. 마이크로비트의 A 버튼을 누르면 디폴트 롤링 이미지가 설정된다.
5. 마이크로비트의 B 버튼을 누르면 마이크로비트의 기울기센서 값(x, y)에 반응하여 도트매트릭스가 연결된 모든 큐브(1, 2번 큐브)에 연결된 도트매트릭스에 롤링디폴트이미지가 반응하여 움직인다.

on start

wait until the first Gcube is connected

wait until 3 Gcubes are connected

on button B pressed

start shifting matrix image for the Gcube 1

on button A pressed

default rolling matrix image :

" _ 0 _ 0 _ 0 _ 0 _ 0 _ 0 _ 0 _ 0 _ "

" _ 0 _ 0 _ 0 _ 0 _ 0 _ 0 _ 0 _ 0 _ "

" _ 0 _ 0 _ 0 _ 1 _ 1 _ 0 _ 0 _ 0 _ 0 _ "

" _ 0 _ 0 _ 1 _ 1 _ 1 _ 1 _ 0 _ 0 _ 0 _ "

" _ 0 _ 0 _ 1 _ 1 _ 1 _ 1 _ 0 _ 0 _ 0 _ "

" _ 0 _ 0 _ 0 _ 1 _ 1 _ 0 _ 0 _ 0 _ 0 _ "

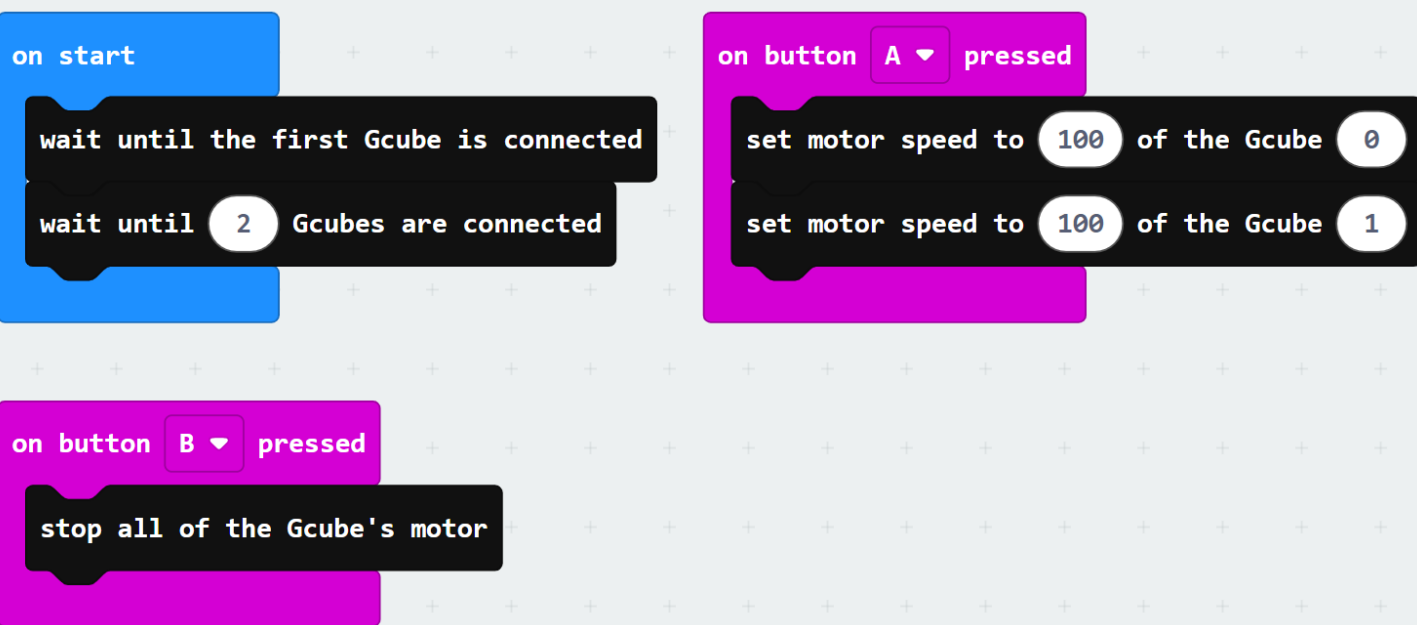
" _ 0 _ 0 _ 0 _ 0 _ 0 _ 0 _ 0 _ 0 _ 0 _ "

" _ 0 _ 0 _ 0 _ 0 _ 0 _ 0 _ 0 _ 0 _ 0 _ "

Example4 – motorSpeed Control

예제 설명

1. 보드에 연결된 0번 G큐브가 켜질 때 까지 (전원버튼 1번 누름) 기다린다.
2. 나머지 1번 G큐브가 연결될 때 (전원버튼 2번 누름) 까지 기다린다.
3. 마이크로비트의 A 버튼을 누르면 0번 1번 G큐브의 모터 속도가 각각 100(최대속도)으로 설정되어 동작한다.
4. 마이크로비트의 B 버튼을 누르면 모든 모터가 멈춘다.



Example5 – motorAngle Control A

예제 설명

1. 보드에 연결된 0번 G큐브가 켜질 때 까지 (전원버튼 1번 누름) 기다린다.
2. 나머지 1번, 2번 G큐브가 연결될 때 (1, 2번 큐브의 전원버튼 2번 누름) 까지 기다린다.
3. 마이크로비트의 A 버튼을 누르면 1번 2번 G큐브의 모터 속도가 각각 시계 방향으로 180도 회전한다.
4. 마이크로비트의 B 버튼을 누르면 1번 2번 G큐브의 모터 속도가 각각 반시계 방향으로 300도 회전한다.

on start

wait until the first Gcube is connected

wait until 3 Gcubes are connected

on button A pressed

set rotation angle to 180 of the Gcube 1

set rotation angle to 180 of the Gcube 2

on button B pressed

set rotation angle to -300 of the Gcube 1

set rotation angle to -300 of the Gcube 2

Example6 – motorAngle Control B

예제 설명

1. 보드에 연결된 0번 G큐브가 켜질 때 까지 (전원버튼 1번 누름) 기다린다.
2. 나머지 1~2번 G큐브가 연결될 때 (각 전원버튼 2번 누름) 까지 기다린다.
3. 마이크로비트의 A 버튼을 누르면 0~2번 G큐브의 모터 속도가 각각 시계 방향으로 180도 회전한다.
4. 마이크로비트의 B 버튼을 누르면 0~2번 G큐브의 모터 속도가 각각 반시계 방향으로 300도 회전한다.

on start

wait until the first Gcube is connected

wait until 3 Gcubes are connected

on button B ▼ pressed

set each Gcube rotation angle

Gcube0 = -300

Gcube1 = -300

Gcube2 = -300

Gcube3 = 0

Gcube4 = 0

Gcube5 = 0

Gcube6 = 0

Gcube7 = 0

on button A ▼ pressed

set each Gcube rotation angle

Gcube0 = 180

Gcube1 = 180

Gcube2 = 180

Gcube3 = 0

Gcube4 = 0

Gcube5 = 0

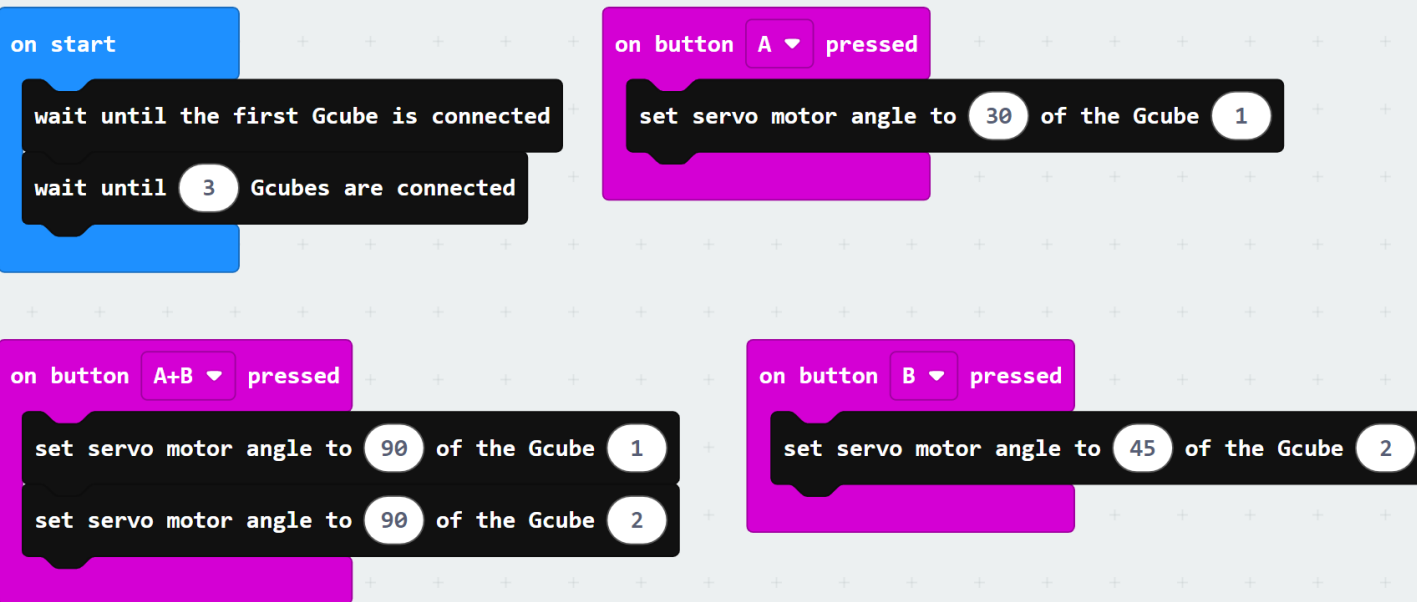
Gcube6 = 0

Gcube7 = 0

Example7 – servomotor Control

예제 설명

1. 보드에 연결된 0번 G큐브가 켜질 때 까지 (전원버튼 1번 누름) 기다린다.
2. 1, 2 번 G큐브가 연결될 때 (각 전원버튼 2번 누름) 까지 기다린다.
3. 마이크로비트의 A 버튼을 1번 G큐브에 붙어 있는 서보모터의 각도가 30도로 돌아간다.
4. 마이크로비트의 A 버튼을 2번 G큐브에 붙어 있는 서보모터의 각도가 45도로 돌아간다.
5. 마이크로비트의 A+B 버튼을 1,2번 G큐브에 붙어 있는 서보모터의 각도가 90도(중앙)으로 돌아간다.

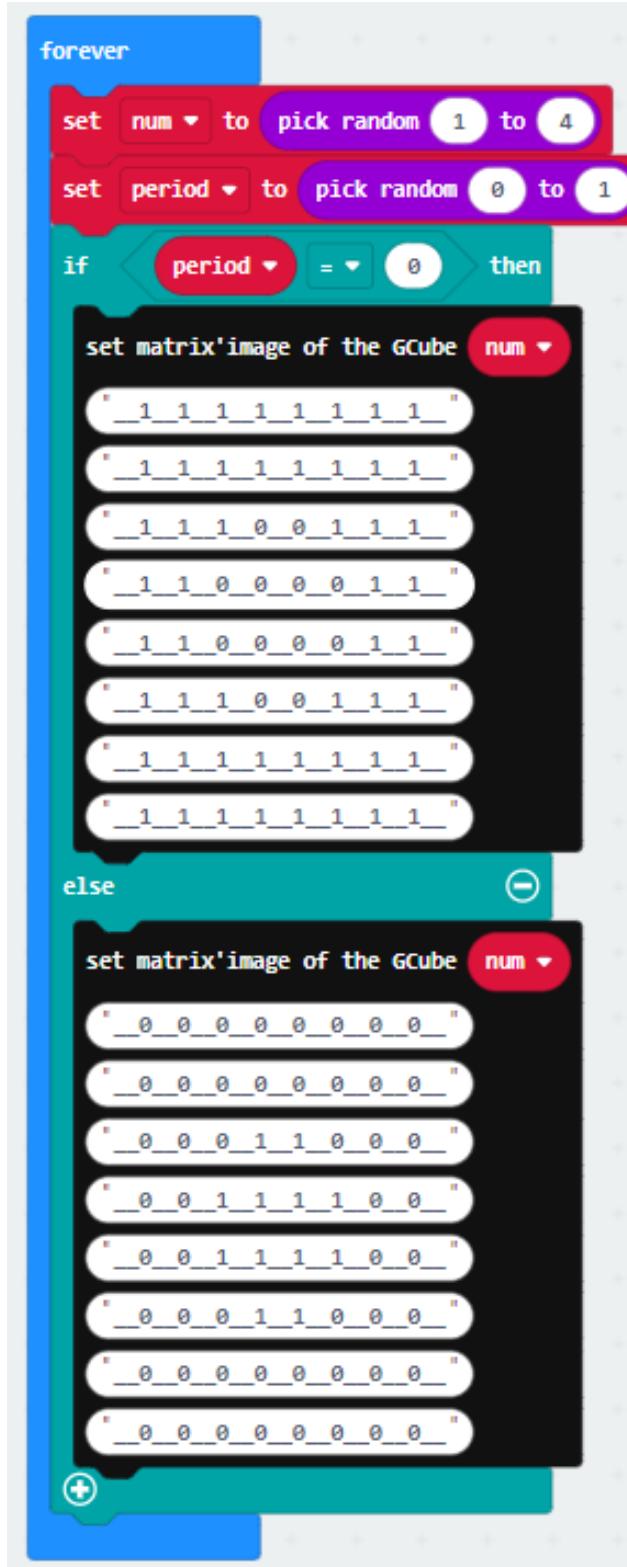


Example8 – Art-Matrix



예제 설명

1. 보드에 연결된 0번 G큐브가 켜질 때 까지 (전원버튼 1번 누름) 기다린다.
2. 나머지 1번, 2번, 3번, 4번 G큐브에 도트 매트릭스를 각각 켜는다.
3. 1, 2, 3, 4 번 G큐브가 연결될 때 (각 전원버튼 2번 누름) 까지 기다린다.
4. 1~4번 큐브가 모두 0번 큐브에 연결되면 4개의 도트매트릭스에 공(Ball) 이미지와 역 이미지가 랜덤하게 선택되어 켜진다.



Example9 – DrawingBot control

예제 설명

1. 핑퐁로봇-드로잉봇을 조립한다.
2. 보드에 연결된 0번 G큐브(펜을 들고 있는 큐브)가 켜질 때 까지 (전원버튼 1번 누름) 기다린다.
3. 1, 2 번 G큐브(바퀴, 양쪽)가 연결될 때 (각 전원버튼 2번 누름) 까지 기다린다.
4. 마이크로비트의 A 버튼을 누르면 드로잉봇이 원을 그리며 회전한다.
5. 마이크로비트의 B 버튼을 누르면 드로잉봇이 멈춘다.
6. 마이크로비트의 A+B 버튼을 누르면 드로잉봇이 ㄷ 자 모양을 펜을 내리고 그린뒤 펜을 올린다.
7. 0번 G큐브를 들어올려 1, 2번 큐브 자동차형 몸체를 마이크로비트의 가속도 센서값에 비례해 조정한다.

```
on start
  wait until the first Gcube is connected
  wait until 3 Gcubes are connected

on button A pressed
  pendown of Drawing Bot
  set each Gcube motor's speed
  Gcube0 = 0
  Gcube1 = -50
  Gcube2 = 100
  Gcube3 = 0
  Gcube4 = 0
  Gcube5 = 0
  Gcube6 = 0
  Gcube7 = 0

on button A+B pressed
  pendown of Drawing Bot
  move PingPong robot 10 cm for drawingbot
  rotate PingPong robot 90 degree for drawingbot
  move PingPong robot 10 cm for drawingbot
  rotate PingPong robot 90 degree for drawingbot
  move PingPong robot 10 cm for drawingbot
  penup of Drawing Bot

on button B pressed
  stop all of the Gcube's motor
  penup of Drawing Bot
```

Example10 – Joystick Car control

on start

wait until the first Gcube is connected

wait until 3 Gcubes are connected

forever

if proximitysensor value of GCube 0 > 200 then

if acceleration (mg) y > 350 then

set Direction to 1

else

set Direction to 0

if acceleration (mg) y < -350 then

set Direction to -1

+

+

if acceleration (mg) x < -350 then

set Left_Gain to 0.7

set Right_Gain to 1

else

set Left_Gain to 0.7

set Right_Gain to 0.7

if acceleration (mg) x > 350 then

set Left_Gain to 1

set Right_Gain to 0.7

+

+

set motor speed to 100 x Direction x Left_Gain of the GCube 1

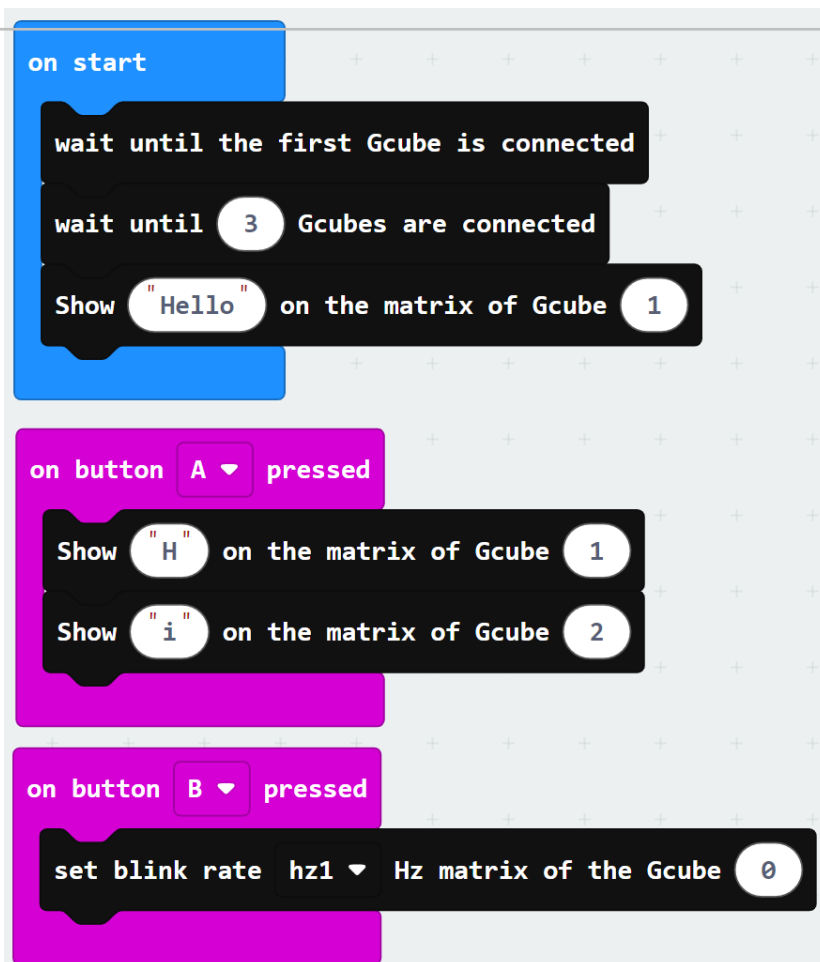
set motor speed to -100 x Direction x Right_Gain of the GCube 2

+

Example11 – dotmatrix blink control

예제 설명

1. 보드에 연결된 0번 G큐브가 켜질 때 까지 (전원버튼 1번 누름) 기다린다.
2. 나머지 1번, 2번 G큐브에 도트매트릭스를 각각 켜는다.
3. 1, 2 번 G큐브가 연결될 때 (각 전원버튼 2번 누름) 까지 기다린다.
4. 마이크로비트의 A 버튼을 누르면 1, 2번 G큐브에 각각 H와 i가 표시된다.
5. 마이크로비트의 B 버튼을 누르면 0이외의 모든 G큐브 (1, 2번)의 도트매트릭스가 1Hz로 깜박이며 이미지를 표시한다.



Library – GCube API 설명

sendGCube(xdata: any[], d_flag: number)

GCube에 시리얼 데이터를 전송합니다.

매개변수:

xdata: any[]: GCube로 전송할 데이터 배열입니다.

d_flag: number: 다음 명령어를 기다릴지 여부를 나타내는 플래그입니다 (선택 사항).

반환: 없음

invValue(a: number)

입력된 숫자의 값을 반전 시킵니다.

매개변수: a: number: 반전시킬 숫자입니다.

반환: number: 반전된 값입니다.

matrixLine(aaa: string)

행렬 라인의 문자열 표현을 숫자 값으로 변환합니다.

매개변수: aaa: string: 행렬 라인을 나타내는 문자열입니다.

반환: number: 도트매트릭스 배열의 숫자 표현값

sendMatrixData(cn: number, t1: number, t2: number, t3: number, t4: number, t5: number, t6: number, t7: number, t8: number)

GCube에 도트매트릭스 표시 데이터를 전송합니다.

매개변수:

cn: number: 큐브 번호입니다.

t1, t2, t3, t4, t5, t6, t7, t8: number: 각 열의 행렬 데이터를 나타내는 숫자 값입니다.

반환: 없음

gripperControl(actionType: gripperStatus)

Ant Bot의 집게(그리퍼) 제어 명령입니다.

매개변수: actionType: gripperStatus: 그리퍼 동작 유형 (열기 또는 닫기).

반환: 없음

leverControl(actionType: leverStatus)

Battle Bot의 레버 제어 명령입니다.

매개변수: actionType: leverStatus: 레버 동작 유형 (내리기 또는 올리기).

반환: 없음

penControl(actionType: penStatus)

Drawing Bot의 펜 제어 명령입니다.

매개변수: actionType: penStatus: 펜 동작 유형 (내리기 또는 올리기).

반환: 없음

rotateWheelRobot(pingpongRobot: robotName, angleValue: number)

Car-type PingPong 로봇 모델의 회전 명령입니다.

매개변수:

pingpongRobot: robotName: 로봇 모델 이름.

angleValue: number: 회전 각도 (예: 90).

반환: 없음

moveWheelRobot(pingpongRobot: robotName, moveLength: number)

Car-type PingPong 로봇 모델의 이동 명령입니다.

매개변수:

pingpongRobot: robotName: 로봇 모델 이름.

moveLength: number: 이동 길이 (예: 10).

반환: 없음

readCubeSensor(cubeNumber: number, sensorSelect: sensorType)

GCube 센서 값을 읽는 명령입니다.

매개변수:

cubeNumber: number: 큐브 번호 (예: 1).

sensorSelect: sensorType: 센서 선택 (근접 센서, 버튼 센서, 외부 포트 센서).

반환: number: 센서 값.

Library – GCube API 설명

`readCubeAccelerometer(cubeNumber: number, axisSelect: cubeAccelerometer)`
GCube의 가속도 센서 값을 읽는 명령입니다.

매개변수:

`cubeNumber: number`: 큐브 번호 (예: 1).
`axisSelect: cubeAccelerometer`: 축 선택 (X 축 또는 Y 축).

반환: `number`: 가속도계 센서 값.

`readCubeArduinoAnalog(cubeNumber: number, portSelect: number)`
GCube의 아두이노 아날로그 포트 값을 읽는 명령입니다.

매개변수:

`cubeNumber: number`: 큐브 번호 (예: 1).
`portSelect: number`: 아날로그 포트 선택 (예: 0).

반환: `number`: 아날로그 포트 센서 값.

`tartShiftingMatrixImage(cubeIndex: number)`
마이크로비트의 가속도계 센서 값에 따라 이미지를 화면이동 시키는 명령입니다.

매개변수: `cubeIndex: number`: 큐브 인덱스 번호 (예: 1).

반환: 없음

`startRollingMatrixImage(duration: number)`
주어진 기간 동안 매트릭스 이미지를 롤링하는 명령입니다.

매개변수: `duration: number`: 시간(회전 횟수) (예: 1000).

반환: 없음

`defaultRollingMatrixImage(t1: string, t2: string, t3: string, t4: string, t5: string, t6: string, t7: string, t8: string)`
8x8 도트 매트릭스의 GCube에 대한 기본 롤링 디스플레이 이미지를 설정하는 명령입니다.

매개변수:

`dm: string`: 더미 인덱스 (예: A).
`t1-t8: string`: 이미지 라인 (예: "0_0_0_0_0_0_0_0").

반환: 없음

`setMatrixDisplay(cn: number, t1: string, t2: string, t3: string, t4: string, t5: string, t6: string, t7: string, t8: string)`
GCube 1 ~ GCube 8의 8x8 도트 매트릭스에 표시 이미지를 설정하는 명령입니다.

매개변수:

`cn: number`: GCube 인덱스 (예: 1).
`t1-t8: string`: 이미지 라인 (예: "0_0_0_0_0_0_0_0").

반환: 없음

`setAllGCubeServoMotorAngle(a0: number, a1: number, a2: number, a3: number, a4: number, a5: number, a6: number, a7: number)`

모든 GCube의 서보 모터 각도를 설정하는 명령입니다.

매개변수:

`dm: string`: 더미 인덱스 (예: A).
`a0-a7: number`: Cube 0 ~ Cube 7의 서보 모터 각도.

반환: 없음

`setAGCubeServoAngle(cubeIndex: number, servoAngle: number)`
특정 GCube의 서보 모터 각도를 설정하는 명령입니다.

매개변수:

`cubeIndex: number`: GCube 번호 (예: 1).
`servoAngle: number`: 서보 모터 각도.

반환: 없음

`setAllGCubeRotationAngle(r0: number, r1: number, r2: number, r3: number, r4: number, r5: number, r6: number, r7: number)`
모든 GCube의 회전 각도를 설정하는 명령입니다.

매개변수:

`dm: string`: 더미 인덱스 (예: A).
`r0-r7: number`: Cube 0 ~ Cube 7의 회전 각도.

반환: 없음

Library – GCube API 설명

setAGCubeRotationAngle(cubeIndex: number, rotationAngle: number)

특정 GCube의 회전 각도를 설정하는 명령입니다.

매개변수:

cubeIndex: number: GCube 번호 (예: 1).

rotationAngle: number: GCube의 회전 각도 (-1000 ~ 1000).

반환: 없음

stopAllGCubeMotor()

모든 GCube의 모터를 정지시키는 명령입니다.

매개변수: 없음

반환: 없음

setAllGCubeSpeed(s0: number, s1: number, s2: number, s3: number, s4: number, s5: number, s6: number, s7: number)

모든 GCube의 모터 속도를 설정하는 명령입니다.

매개변수:

dm: string: 더미 인덱스 (예: A).

s0-s7: number: Cube 0 ~ Cube 7의 모터 속도.

반환: 없음

setAGCubeSpeed(cubeIndex: number, motorSpeed: number)

특정 GCube의 모터 속도를 설정하는 명령입니다.

매개변수:

cubeIndex: number: GCube 번호 (예: 1).

motorSpeed: number: GCube의 모터 속도.

반환: 없음

waitAllGCubesConnect(cnumber: number)

사용자가 설정한 숫자의 GCube가 연결될 때까지 대기하는 명령입니다.

매개변수: cnumber: number: 연결을 기다리는 GCube의 수 (예: 2).

반환: 없음

waitFirstGCubeConnect()

첫 번째 GCube가 보드에 연결될 때까지 대기하는 명령입니다.

매개변수: 없음

반환: 없음